

GLUE-LP: um protocolo executável para avaliar predição de ligações com GNN sem entregar a aresta-alvo ao modelo

Graph Link-evaluation Under Exclusion

(Título disambiguado do benchmark GLUE de PLN — Wang et al., 2018 — com o qual este projeto não tem relação.)

Autores: [placeholder — nomes] **Afiliação:** [placeholder — instituição / laboratório / grupo]
Contato: [placeholder — email] **Versão do artefato:** 0.5.0 · **Idioma principal:** pt-BR · **Abstract:** English **Tipo:** manuscrito de Design Science (IMRaD) acompanhado do pacote reproduzível glue_lp

Integridade experimental. Todos os AUC, t de Student pareado, desvios-padrão e Δ citados neste manuscrito foram lidos de `experiments/*.json`, `experiments/RESULTADOS.md` ou do apêndice medido da escada de vazamento. O que não foi medido está marcado explicitamente como **proposto** / **não medido** ou **futuro trabalho**. Não inventamos números. Não afirmamos SOTA. OGB não rodou.

Banner de integridade (pré-correção). Todos os AUC, t e Δ de Cora / Citeseer / Pubmed reportados neste manuscrito foram medidos *sob o código anterior à correção completa do early stopping simétrico em `train_encoder`` (score de validação efetivamente morto / assimetria valid vs leaky). O `assert Q+ ∩ Emp = ∅` e o harness já estavam presentes; o critério de parada não. Não reivindicamos resultados pós-correção. Re-execução completa = trabalho futuro prioritário. Números da escada L1/L3/L4 e da enumeração exaustiva no tech tree sintético (NumPy) não dependem desse bug* de treino PyTorch.

Resumo

A predição de ligações (*link prediction*) pergunta, dado um grafo incompleto, quais pares de vértices deveriam ser aresta. Em recomendação, em redes de citação científica e em árvores de *crafting* de jogos, a pergunta é a mesma; o instrumento moderno é uma rede neural de grafos (GNN). O atalho moderno — e não raro o *default* de tutoriais que reutilizam o objeto `Data` do classificador de nós — é deixar o par alvo (u,v) no índice de *message passing* enquanto se pede ao modelo que adivinhe (u,v) . O encoder vê o rótulo. A métrica sobe. O artigo parece melhor do que o sistema em produção, onde a aresta futura ainda não existe.

Zhu et al. (SpotTarget) e Li et al. (HeaRT) descreveram o diagnóstico com fôlego de *benchmark*. A decisão de desenho deste artefato é outra: a exclusão da aresta-alvo não é um parágrafo de *related work* nem um *flag* silencioso — é um **assert** que quebra o processo. Formalmente, seja E_{mp} o conjunto de arestas visíveis ao *message passing* e Q^{+} o conjunto de positivos de avaliação. O protocolo **válido** exige $Q^{+} \cap E_{mp} = \emptyset$. O protocolo **leaky** admite $Q^{+} \subseteq E_{mp}$ como controle do atalho. O relatório sem regime declarado não conta. Heurísticas clássicas (vizinhos comuns, Adamic–Adar, *preferential attachment*) entram no mesmo Q , ou o *run* é incompleto.

Série 1 (GCN denso, 50 épocas fixas, Cora, 5 seeds, negativos 1:1): AUC 0.708 ± 0.020 no protocolo válido contra 0.826 ± 0.019 no leaky ($\Delta = +0.118$). Negativos hard por vizinhos comuns: GCN 0.619 ± 0.009 . Adamic–Adar (AUC Mann–Whitney com empate = $\frac{1}{2}$): 0.720 ± 0.007 no uniforme e 0.542 ± 0.016 no hard. Split indutivo (15% dos nós ocultos): GCN 0.725 ± 0.012 ; CN/AA caem à chance (0.496 ± 0.001).

Série 2 (GCN esparso + *early stopping*, paciência 12): Cora válido 0.728 ± 0.020 vs leaky 0.782 ± 0.046 , t pareado = 2.61 ($gl = 4$; a seed 4 inverte o sinal; o limiar bilateral 5% $\approx 2,78$ **não** é atravessado). Citeseer válido 0.733 ± 0.029 vs leaky 0.833 ± 0.030 , $t = 5.77$ (Δ médio $+0.100$). GraphSAGE (média plena, Cora válido): 0.577 ± 0.025 .

Onda 3: GAT (1 cabeça) em Cora — válido 0.782 ± 0.025 vs leaky 0.876 ± 0.021 , $t = 20.4$. GCN com negativos *degree-matched*: 0.611 ± 0.033 . Pubmed (GCN esparso, 3 seeds): válido 0.886 ± 0.003 vs leaky 0.921 ± 0.009 , $t = 7.75$ ($gl = 2$).

Escada de vazamento no *tech tree* sintético (12 nós, 10 seeds, NumPy): L1 0.7133 ± 0.1726 , L3 0.7789 ± 0.1976 , L4 (meia-aresta) 0.8602 ± 0.0915 . *Preferential attachment* sobre $\Delta(L3-L1) = +0.465$. Um bug silencioso de meia-aresta (L4) pode inflar a AUC tanto quanto ou mais que o vazamento completo (L3).

Não é SOTA. **OGB não rodou**. Não propomos nova camada de convolução — propomos um *harness* de Design Science que recusa o atalho: a exclusão vira invariante executável, o mesmo Q alimenta GNN e heurísticas, e o CI fica vermelho quando alguém esquece.

Palavras-chave: GNN; predição de ligações; vazamento (*leakage*); Cora; Citeseer; Pubmed; avaliação indutiva; Design Science; protocolo executável; Adamic–Adar; GraphSAGE; GAT; VGAE.

Abstract

GLUE-LP (*Graph Link-evaluation Under Exclusion*) is a Design Science artifact that makes target-edge exclusion an unskippable invariant of link-prediction evaluation with graph neural networks: $Q^{+} \cap E_{mp} = \emptyset$ in valid mode, enforced by an assert that aborts the run. The same evaluation set Q is scored by GCN, GraphSAGE, GAT and classical heuristics (common neighbors, Adamic–Adar, preferential attachment) under uniform, hard-CN and degree-matched negative sampling, with random, inductive-node and synthetic-temporal splits.

On Cora with a fixed-epoch dense GCN (Series 1, 5 seeds) the leaky protocol adds +0.118 AUC (0.708 ± 0.020 valid vs 0.826 ± 0.019 leaky). A second series with sparse GCN and early stopping replicates the direction on Citeseer (paired $t = 5.77$, $\Delta = +0.100$) but is only suggestive on Cora ($t = 2.61$, $df = 4$; one seed inverts). GAT on Cora yields $t = 20.4$; Pubmed GCN (3 seeds) yields $t = 7.75$. A synthetic leakage ladder (L1/L3/L4, 10 seeds) shows that a silent half-edge bug (L4) can inflate AUC as much as full leakage (L3), while preferential attachment is far more sensitive ($\Delta = +0.465$) than CN/AA. No OGB runs. No new message-passing layer. All numbers come from measured experiments/* .json; unmeasured items are marked as proposed.

Keywords: GNN; link prediction; leakage; Cora; Citeseer; Pubmed; inductive evaluation; design science; executable protocol; Adamic–Adar; GraphSAGE; GAT.

1. Introdução

1.1 Problema e motivações

A tarefa de predição de ligações é antiga na ciência de redes. Liben-Nowell e Kleinberg (2007) a sistematizaram em redes sociais; Newman (2001) e Barabási e Albert (1999) ligaram crescimento preferencial a padrões observáveis de grau; Adamic e Adar (2003) ponderaram vizinhos comuns pela raridade. O que mudou na última década não foi a pergunta — *quais pares deveriam ser aresta?* — e sim o instrumento: *message passing* aprendido (Gilmer et al., 2017; Kipf e Welling, 2017; Hamilton et al., 2017; Veličković et al., 2018) e o decoder bilinear popularizado por autoencoders de grafo (Kipf e Welling, 2016, VGAE/GAE).

Três domínios motivam o artefato. **Citação científica:** Cora, Citeseer e Pubmed (Sen et al., 2008, via LINQS) são o *Planetoid* clássico: documentos como nós, citações como arestas, *bag-of-words* como atributos. Prever uma citação futura é predição de ligação; o modelo em produção não pode ter visto a aresta que ainda não existe. **Recomendação:** o mesmo encoder–decoder em grafos bipartidos usuário–item (Berg, Kipf e Welling, 2017, GC-MC) enfrenta a mesma tensão entre *message passing* e rótulo. **Tech tree / crafting:** em jogos e sistemas de progressão, a árvore de receitas é um grafo pequeno com *timestamps* naturais — domínio em que *split* temporal é honesto sem inventar relógio em LINQS. O pacote inclui um *tech tree* sintético (12 nós, 20 arestas) exatamente para essa ilustração.

O problema de engenharia que este manuscrito ataca não é *qual GNN é SOTA*, e sim *como garantir que a avaliação não entregue o rótulo ao encoder*. Formalmente:

$$Q^+ \cap E_{\text{mp}} = \emptyset \text{ quad (modo válido).}$$

Quando a interseção não é vazia, o encoder pode propagar informação do próprio rótulo para a embedding usada pelo decoder. A AUC Mann–Whitney sobe. O *paper* reporta um número que o sistema em produção — onde (u,v) ainda não está no grafo — não reproduz. Não é necessariamente malícia: é o *default* de um *notebook* que reutiliza `edge_index` completo e mascara apenas o decoder.

1.2 Lacuna: diagnóstico sem harness incontornável

A literatura recente de avaliação já documentou o problema. Li et al. (2023, HeaRT, arXiv:2306.10453) argumentam que políticas de negativos e *splits* inadequados distorcem *rankings* de GNN para *link prediction*. Zhu et al. (2024, SpotTarget, WSDM / arXiv:2306.00899) isolam *pitfalls* incluindo vazamento de aresta-alvo. Hu et al. (2020, OGB) padronizam *splits* oficiais e Hits@K — padrão que este artefato **não executou** (marcado como proposto / não medido). Zhang e Chen (2018, SEAL) mostram que *subgraphs enclosing* são poderosos, mas estão fora do núcleo medido aqui.

A lacuna que GLUE-LP preenche não é *mais um paper de diagnóstico*, e sim um **artefato executável** no sentido de Design Science (Hevner et al., 2004; Peffers et al., 2007): protocolo + código + invariante + evidência medida + checklist de reprodutibilidade. A exclusão deixa de ser conselho e vira `AssertionError` no CI.

1.3 Contribuições

Enquadramos o trabalho como *Design Science Research* em engenharia de ML / sistemas de informação: o artefato é o protocolo `glue_lp`; a avaliação é a série de experimentos medidos; a utilidade é tornar o atalho *incontornável* no processo de desenvolvimento.

- **Protocolo executável** com modos `valid` e `leaky`, invariante $Q^+ \cap E_{\text{mp}} = \emptyset$ no modo válido, e relatório incompleto se heurísticas clássicas não compartilham o mesmo Q .
- **Harness único** para GCN, GraphSAGE e GAT (decoder bilinear) e para CN / Adamic–Adar / preferential attachment, com políticas de negativos (uniforme, hard-CN, *degree-matched*) e *splits* aleatório, indutivo e temporal sintético.
- **Evidência medida** em Cora, Citeseer e Pubmed (séries 1–2, onda 3) mais escada L1/L3/L4 e enumeração exaustiva no *tech tree* sintético — todos os números em `experiments/*.json`.
- **Pacote reproduzível** (JSON versionados, figuras fig1–fig9, testes pytest em CI sem GPU, manuscrito DOCX/PDF gerado por script, API opcional somente leitura).

1.4 Hipóteses (honestas)

Formulamos hipóteses *a priori* e reportamos o que os dados medidos permitem concluir — inclusive quando a conclusão é *sugestiva* e não *definitiva*.

- **H1 (inflação leaky)**. O protocolo `leaky` infla a AUC do GNN relativamente ao válido no mesmo *seed* e no mesmo Q^- . *Esperado*: efeito positivo e estável. *Observado*: claro na série 1 (Cora, $\Delta = +0.118$, cinco seeds no mesmo sentido), em Citeseer ($t = 5.77$), em GAT-Cora ($t = 20.4$) e em Pubmed ($t = 7.75$); apenas *sugestivo* em Cora com *early stopping* ($t = 2.61 < 2.78$; seed 4 inverte).
- **H2 (não-uniformidade entre scorers)**. O vazamento **não** é uniforme: *preferential attachment* é mais sensível que CN/AA porque o grau dos *endpoints* sobe **somente** nos positivos quando estes entram em E_{mp} . *Observado no sintético*: $\Delta(L3-L1)$ PA = $+0.465$ vs AA $+0.199$ e CN $+0.164$. **Proposto / não medido** como confirmação formal da mesma proposição em Cora/Citeseer/Pubmed sob a escada L.
- **H3 (política de negativos troca o ranking)**. Em Cora válido, a política de negativos altera quem vence entre GCN e Adamic–Adar. *Observado*: no uniforme, AA (0.720 ± 0.007) $\approx$ GCN (0.708 ± 0.020); no hard, GCN (0.619 ± 0.009) $>$ AA (0.542 ± 0.016). Quem publica só o uniforme escolhe o Q mais complacente.

Não afirmamos que hidden=32 é ótimo. Não afirmamos que GAT de uma cabeça representa atenção em geral. Afirmamos que o atalho é mensurável, que sua visibilidade estatística depende de corpus / critério de parada / encoder, e que o *harness* torna essa mensuração *incontornável*.

1.5 Organização do manuscrito

A Seção 2 posiciona o trabalho frente à literatura clássica e moderna. A Seção 3 formaliza LP, leakage e a escada L0–L4. A Seção 4 descreve o artefato de Design Science. A Seção 5 detalha a *stack* tecnológica (medido vs proposto). A Seção 6 dá o protocolo experimental completo. A Seção 7 reporta e discute todos os resultados medidos. As Seções 8–11 cobrem ameaças à validade, limitações, reprodutibilidade e conclusão. Apêndices trazem revisão técnica, mapa de módulos, tabelas JSON, glossário e a ameaça do *bug* histórico de *early stopping*.

2. Trabalhos relacionados

2.1 Predição de ligações clássica

Liben-Nowell e Kleinberg (2007) formularam o *link-prediction problem* para redes sociais e compararam índices de vizinhança e caminhos. O trabalho permanece a referência conceitual: a tarefa é ranquear pares não observados; a avaliação deve separar treino e teste de forma que o futuro não vaze para o passado. Adamic e Adar (2003) introduziram a ponderação de vizinhos comuns por $1/\log \deg(w)$, capturando a intuição de que vizinhos raros carregam mais evidência. Newman (2001) estudou *clustering* e *preferential attachment* em redes de coautoria; Barabási e Albert (1999) derivaram a lei de potência de grau a partir do crescimento preferencial. No *harness* GLUE-LP, CN, Adamic–Adar e preferential attachment **não são opcionais**: compartilham o mesmo Q do GNN, ou a célula experimental é incompleta.

Essa insistência tem motivo empírico. Na série 1 (Cora, negativos uniformes), Adamic–Adar com AUC de empate = $\frac{1}{2}$ marca 0.720 ± 0.007 , numericamente lado a lado do GCN válido (0.708 ± 0.020). Omitir a heurística no mesmo Q ocultaria que o GNN, nessa célula, não ganha de um índice de 2003. No regime hard, a história inverte — e omitir o hard ocultaria a outra metade.

2.2 Message passing e GNN para arestas

Gilmer et al. (2017) formalizaram *message passing neural networks* (MPNN) como framework unificador: mensagens $m_v\{t\}$ agregadas de vizinhos, atualização de estado, *readout*. Kipf e Welling (2017, arXiv:1609.02907, ICLR) propuseram o GCN com renormalização $\{A\} = D^{-1/2} (A+I) D^{-1/2}$ para classificação semi-supervisionada de nós. **Para link prediction, a referência metodológica principal deste harness é outra:* Kipf e Welling (2016, VGAE/GAE, arXiv:1611.07308), que treinam um encoder GCN e um decoder de produto interno (ou bilinear) sobre pares. Distinguir os dois papers evita o erro comum de citar só o ICLR 2017 quando o objeto é aresta.

Hamilton et al. (2017, GraphSAGE, arXiv:1706.02216, NeurIPS) introduziram agregação amostrada e concatenação $[h_v, \text{AGG}(N(v))]$ para aprendizado indutivo. Neste pacote medimos a variante de média plena (sem amostragem de vizinhos) — **GraphSAGE com amostragem permanece proposto / não medido**. Veličković et al. (2018, GAT, arXiv:1710.10903, ICLR) substituem a agregação uniforme por atenção; medimos GAT de **uma cabeça** (multi-head: proposto / não medido). Zhang e Chen (2018, SEAL, arXiv:1802.09691, NeurIPS) argumentam que *enclosing subgraphs* com rotulagem de Double-Radius Node Labeling superam produto interno puro — extensão natural, **não medida** aqui. Berg, Kipf e Welling (2017, GC-MC) ilustram decoders bilineares em grafos bipartidos; nosso decoder permanece $s(u,v) = \{z_u, z_v\}$.

2.3 Avaliação com e sem vazamento

Li et al. (2023, HeaRT, arXiv:2306.10453) mostram que *rankings* de métodos de *link prediction* mudam drasticamente sob *hard negative sampling* estruturado e *splits* mais realistas. Zhu et al. (2024,

SpotTarget) catalogam *pitfalls* de GNN-LP, incluindo o vazamento de aresta-alvo no *message passing*. GLUE-LP opera na mesma família de preocupação, mas com ênfase de Design Science: a exclusão é assert, o leaky é controle explícito (nunca o *headline*), e o mesmo Q alimenta heurísticas.

Hu et al. (2020, Open Graph Benchmark, arXiv:2005.00687) padronizam datasets `ogbl-*`, *splits* oficiais e Hits@K / MRR. **OGB não foi executado neste artefato**; `integrations/ogb_loader.py` é stub. Sen et al. (2008) disponibilizam Cora, Citeseer e Pubmed via LINQS — os três corpora medidos. O *Planetoid* de Yang, Cohen e Salakhutdinov (2016) popularizou os *splits* de classificação de nós; para LP, reutilizamos os grafos e atributos, com *holdout* de arestas próprio do protocolo (10% teste, 5% validação).

2.4 Design Science e artefatos de avaliação

Hevner et al. (2004) definem Design Science em sistemas de informação como ciclo de construção e avaliação de artefatos que resolvem problemas de classe. Peffers et al. (2007) propõem metodologia em etapas (identificação do problema, objetivos, *design*, demonstração, avaliação, comunicação). GLUE-LP trata o protocolo como artefato: relevância (tutoriais vazam; produção não pode), rigor (assert + JSON + testes), utilidade (CI sem GPU; *runners* determinísticos por seed). Não confundimos o artefato com o *leaderboard*: a contribuição é a invariante, não um novo encoder.

2.5 Posicionamento

Em relação a HeaRT e SpotTarget, somos mais estreitos em *benchmark* (LINQS + sintético; sem OGB) e mais rígidos em engenharia (assert, mesmo Q, modos explícitos). Em relação a VGAE/GAE, reutilizamos o encoder-decoder bilinear sem reivindicar novelty arquitetural. Em relação a SEAL, apontamos a extensão e a marcamos como não medida. Em relação ao GLUE de PLN (Wang et al., 2018), a homonímia é acidental: nosso acrônimo é *Graph Link-evaluation Under Exclusion*.

2.6 Surveys, pitfalls de avaliação e LP-GNN recentes

Lü e Zhou (2011) sistematizam *link prediction* em redes complexas (índices de similaridade, paths, máxima verossimilhança). Shchur et al. (2018) e Errica et al. (2020) documentam *pitfalls* de avaliação de GNN (*splits*, seeds, vazamentos de protocolo). Dwivedi et al. (2023, JMLR) propõem *benchmarks* controlados de GNN. Kapoor e Narayanan (2023) generalizam a crise de *leakage* em ciência baseada em ML. Yang, Cohen e Salakhutdinov (2016, Planetoid) popularizam embeddings semi-supervisionados nos grafos LINQS. Grover e Leskovec (2016, node2vec) e He et al. (2020, LightGCN) ilustram a linha embedding/recomendação. Yun et al. (2021, Neo-GNN), Chamberlain et al. (2023, BUDDY) e Wang, Yang e Zhang (2024, NCNC) avançam LP-GNN estrutural — **fora do núcleo medido** aqui (proposto como *baseline* futuro). Fey e Lenssen (2019, PyG) são a *stack* de fato da comunidade; nosso núcleo medido é PyTorch puro + stubs. Pineau et al. (2021) e o checklist ACM/NeurIPS informam a Seção 10. Gregor e Hevner (2013) posicionam comunicação de DSR.

2.7 Tabela de diferenciação (SpotTarget / HeaRT / OGB / GLUE-LP)

Dimensão	SpotTarget (Zhu et al., WSDM 2024)	HeaRT (Li et al., 2023)	OGB (Hu et al., 2020)	**GLUE-LP**
Foco	*pitfalls* (incl. target-link)	negativos / *splits* duros	*benchmark* Hits@K	protocolo + assert DSR
Exclusão alvo	análise empírica	boa prática	*split* oficial	**assert de processo**
Modo leaky	estudo do impacto	—	—	controle explícito
Heurísticas mesmo Q	parcial	—	—	obrigatório
Escada L4 meia-aresta	—	—	—	**medida (sintético)**
OGB executado	discute	discute	**define**	**não**
Autoria correta	**Zhu, Zhou, Ioannidis, Qian, Ai, Song, Koutra**	Li et al.	Hu et al.	este artefato

3. Fundamentação formal

3.1 Predição de ligações

Seja $G = (V, E)$ um grafo simples não dirigido (após simetrização das citações LINQS), com atributos de nó $X \in \mathbb{R}^n \times d$. Um *split* de arestas particiona E em treino E_{train} , validação E_{val} e teste E_{test} . Os positivos de avaliação são $Q^+ = E_{\text{test}}$ (ou o subconjunto indutivo correspondente). Os negativos Q^- são pares não-aresta amostrados sob uma política declarada. O conjunto de avaliação é $Q = Q^+ \cup Q^-$.

Um encoder f_θ produz embeddings $z_i = f_\theta(X, E_{\text{mp}})_i$, onde $E_{\text{mp}} \subseteq E$ é o conjunto de arestas visíveis ao *message passing*. O decoder escoreia pares $s(u, v) = \langle z_u, z_v \rangle$ (produto interno; bilinear diagonal). O treino minimiza BCE com logits em positivos de treino contra negativos amostrados (1:1 nas séries medidas). A métrica principal de avaliação é a AUC Mann–Whitney em Q , com empate contado como $\frac{1}{2}$ (implementação em `glue_lp.metrics.roc_auc`). Métricas auxiliares já presentes no código — *average precision* (AP), Hits@K e MRR — são registradas nos JSON da série 1; o *headline* do manuscrito permanece a AUC porque é a série completa e comparável entre GNN e heurísticas.

3.2 Definição de target-edge leakage

Definição (leakage de aresta-alvo). Dizemos que há *target-edge leakage* quando existe $e \in Q^+$ tal que $e \in E_{\text{mp}}$ (ou a versão dirigida correspondente após simetrização incompleta). No modo `valid`, o harness exige a negação dessa condição via `assert_no_leakage` (`glue_lp.protocol`). No modo `leaky`, a interseção é intencional e documentada: $E_{\text{mp}} = E_{\text{train}} \cup E_{\text{val}} \cup E_{\text{test}}$ (nas células medidas).

A definição é operacional, não metafísica. Ela não cobre todos os *leakages* imagináveis (por exemplo, vazamento via hiperparâmetros escolhidos no teste), mas cobre o atalho estrutural que tutoriais reproduzem: o encoder agrega sobre a aresta que o decoder deveria adivinhar.

3.3 Escada L0–L4

Para tornar o vazamento *graduável* — e não um booleano único — adotamos uma escada operacional. Os níveis L1, L3 e L4 foram **medidos** no *tech tree* sintético (`leakage_ladder_resumo.json`); L0 e L2 são propostos / não medidos na escada JSON.

Nível	Definição operacional	Status neste pacote
L0	Conceito / <i>*checklist*</i> sem mp especial; documentação apenas	proposto
L1	$(E_{\text{mp}} = E_{\text{train}}) (*\text{valid}*)$	**medido** (sintético)
L2	$(E_{\text{mp}} = E_{\text{train}} \cup E_{\text{val}})$ (intermediário)	proposto / não medido na escada JSON
L3	$(E_{\text{mp}} = E_{\text{train}} \cup Q^+) (*\text{leaky}* \text{ pleno de positivos})$	**medido**
L4	$(E_{\text{mp}} = E_{\text{train}} \cup \text{meia-aresta (uma direção do alvo)})$	**medido**

L4 modela um bug frequente ao portar código dirigido $\leftrightarrow$ não dirigido: simetrização incompleta. Qualitativamente, L4 **não** é leakage "pequena": no sintético, a média L4 superou L3 (0.8602 ± 0.0915 vs 0.7789 ± 0.1976), com menor desvio-padrão. Proposição qualitativa: *bugs silenciosos de meia-aresta podem inflar a métrica tanto quanto o vazamento completo, às vezes com variância menor* — argumentada pelos números medidos da escada, sem extrapolar para LINQS sem medição.

3.4 Proposições qualitativas sobre leakage (P1–P3)

P-PA / P1 (não-uniformidade). O deslocamento Δ AUC sob vazamento não é uniforme entre scorers. Preferential attachment escoreia $\deg(u) \cdot \deg(v)$. Incluir positivos em E_{mp} aumenta os graus dos *endpoints* **somente** nos pares positivos — um *bump* aditivo exclusivo de Q^+ . CN e AA dependem de

vizinhos comuns; o efeito de primeira ordem em Q amostrado pode ser menor, com efeitos de segunda ordem via grau dos intermediários. **P1 confirmada no tech tree:** $\Delta(L3-L1)$ PA = +0.4648, AA = +0.1992, CN = +0.1641. Reexecução da escada em Cora/Citeseer/Pubmed: **futuro trabalho.**

P-leak / P2 (visibilidade estatística depende do protocolo de treino). O mesmo atalho estrutural pode ser *claro* ou *sugestivo* conforme o critério de parada e o corpus. Evidência: série 1 (épocas fixas) vs série 2 (*early stopping*) em Cora — ver §7. Não tratamos P2 como lei universal; tratamos como achado de laboratório que desarma a frase "H1 sempre confirmada".

P3 (atenção não imuniza; corolário de P-leak). Se $(u,v) \in E_{mp}$, o mecanismo de atenção pode *aumentar* o peso da mensagem no canal do rótulo. Evidência medida: GAT-Cora $t = 20.4$. Não afirmamos que multi-head se comporta igual (não medido).

3.5 Métricas: Mann–Whitney, AP, Hits@K, MRR

A AUC implementada é a probabilidade de um positivo ranqueado acima de um negativo, com empates como meio. Isso importa para heurísticas: CN e AA produzem muitos empates em zero; uma AUC que ignore empates ou que use ordenação estável arbitraria infla o número. A série 1 gravou AUC "ingênua" $\sim 1,0$ para CN/AA no JSON bruto; `heuristics_rescored.json` recalcula com empate = $\frac{1}{2}$ — **estes** são os números do manuscrito (0.720 ± 0.007 no uniforme válido). AP, Hits@3, Hits@10 e MRR existem em `glue_lp.metrics` e no JSON da série 1; não os usamos como *headline* porque (i) Hits@K em amostragem 1:1 não é Hits@K OGB e (ii) a comparação GNN $\leftrightarrow$ heurística é mais limpa em AUC Mann–Whitney.

4. Método / artefato de Design Science

4.1 Requisitos do artefato

Derivamos requisitos a partir do problema (Seção 1) e do enquadramento Hevner/Pefferers:

- **R1 (invariante).** No modo válido, $Q^+ \cap E_{mp} = \emptyset$ é assert, não documentação.
- **R2 (controle leaky).** O modo leaky existe, é explícito, e nunca é o número de *abstract* sem o par válido.
- **R3 (mesmo Q).** GNN e heurísticas CN/AA/PA escoreiam o mesmo Q na mesma célula.
- **R4 (políticas de negativos declaradas).** Uniforme, hard-CN e *degree-matched* são primeira classe.
- **R5 (splits declarados).** Aleatório, indutivo (nós ocultos) e temporal (sintético) são suportados; LINQS não tem *timestamp* de aresta nativo.
- **R6 (reprodutibilidade).** Seeds fixas, TrainConfig congelado, JSON versionado, testes CI sem GPU.
- **R7 (honestidade).** O que não rodou (OGB, PyG, DGL, Optuna, W&B, SEAL, GAT multi-head, SAGE amostrado) está marcado como proposto / não medido.

4.2 Harness e invariante

```

1. data ← load_cora | load_citeseer | load_pubmed | crafting_graph
2. split ← random_edge_split | inductive_node_split | split_graph
3. mp ← train                se modo = valid
   mp ← train ∪ val ∪ test   se modo = leaky
4. se valid e  $(Q^+ \cap mp) \neq \emptyset$ : raise AssertionError
5. z ← train_encoder(X, mp, ...) # early stopping por AUC de val real
6. Q- ← uniforme | hard-CN | degree-matched
7. AUC(GNN, Q) e AUC(CN/AA/PA, mesmo Q)
8. gravar célula em experiments/*.json
```

A máscara não é comentário: `mp_edges` é argumento de `train_encoder` / `split_graph`. `assert_no_leakage` quebra o processo antes do treino. Testes em `tests/test_invariante.py` e `tests/test_leakage_ladder_smoke.py` cobrem o caminho feliz e o caminho que deve falhar.

4.3 Splits

Aleatório (transdutivo de arestas). Holdout 10% teste / 5% validação / resto treino, seed no gerador NumPy (`ProtocolConfig.test_frac=0.10, val_frac=0.05`). Usado nas séries 1–2 e onda 3 (exceto indutivo).

Indutivo (nós ocultos). 15% dos nós são removidos do grafo de treino (`inductive_node_frac=0.15`); positivos de teste são arestas que tocam nós ocultos. Medido em Cora (série 1). Interpretação: heurísticas de vizinhança perdem o suporte local; o GCN ainda lê *bag-of-words* via self-loop — não é milagre espectral.

Temporal sintético. O *tech tree* (`graph.crafting_graph`) carrega *timestamps* de aresta; `protocol.split_graph` corta no tempo. LINQS não oferece relógio de aresta — não inventamos.

4.4 Negative sampling

- **Uniforme 1:1:** um negativo por positivo, amostrado uniformemente entre não-arestas. Célula mais complacente.
- **Hard-CN:** negativos escolhidos entre pares de alto *common neighbors* (difíceis para índices de vizinhança). Célula que troca o ranking GCN $\leftrightarrow$ AA.
- **Degree-matched:** para cada positivo (u,v), amostra-se w com $|\deg(w)-\deg(v)| \leq 2$ (implementação do pacote). Alinha empiricamente com hard-CN no Cora válido (0.611 ± 0.033 vs 0.619 ± 0.009).

4.5 Encoders e decoder

- **GCN** (série 1: A denso; série 2 / onda 3: A esparso via `index_add_`): duas camadas, hidden 32, ReLU + dropout 0,5 na oculta, saída linear. Renormalização $\{A\} = D^{-1/2}(A+I)D^{-1/2}$.
- **GraphSAGE:** concat $[h_v, \text{mean}(N(v))]$; média plena (sem amostragem). Saída com ``F.normalize`` L2 aplicada no código como correção recomendada pelo paper; a AUC 0.577 ± 0.025 foi medida **antes** desta normalização — re-run **não medido**.
- **GAT:** uma cabeça, LeakyReLU(0,2), ELU, dropout de atenção 0,4. Multi-head: proposto / não medido.
- **Decoder:** produto interno $\langle z_u, z_v \rangle$.
- **Otimização:** Adam, lr = 0,01, weight decay $5 \cdot 10^{-4}$, BCE com logits, 1 negativo de treino por positivo.

Série 1: 50 épocas fixas, sem *early stopping*. Série 2 / onda 3: *early stopping* por AUC de validação **real** (paciência 12; teto 70 épocas na série 2, 40 na onda 3 Cora, 25 no Pubmed). **Nota de integridade:** um *bug* histórico fazia `score = 0.0` sempre que `pos_val` era fornecido, matando o *early stopping* no modo válido (ver Apêndice E). O código foi corrigido; os números publicados de Cora/Citeseer/Pubmed foram medidos sob o código *anterior* à correção completa do critério simétrico. Re-run: **futuro trabalho** — não substituímos JSON à mão.

4.6 Domínio ilustrativo (tech tree)

O grafo de *crafting* (`crafting_graph`) tem 12 nós e 20 arestas com *timestamps*. Serve a três papéis: (i) *split* temporal honesto; (ii) escada L1/L3/L4 barata em CPU; (iii) enumeração exaustiva de pares ($\text{binom}\{12\}\{2\} = 66$, ~ 54 avaliados após remover treino). Não é o estudo principal — é o laboratório de *leakage* controlado.

4.7 Critérios de avaliação do artefato (Design Science)

O mapa completo DSRM (Pefferers) e guidelines (Hevner), com separação explícita entre critérios do **artefato** e do **fenômeno**, está no **Apêndice D**. Resumo: o artefato é o protocolo/harness; o fenômeno é o Δ AUC sob leakage. Avaliamos ambos, sem confundir *leaderboard* com fidelidade da invariante.

4.7b Critérios de avaliação do artefato (continuidade)

Seguindo Hevner, avaliamos o artefato por: **fidelidade** (o assert reproduz a definição formal), **completude** (modos, splits, negativos, heurísticas), **usabilidade** (CI sem GPU; `check_protocol.py`), **impacto epistêmico** (séries medidas que mostram o atalho e seus limites estatísticos). Não avaliamos por posição em *leaderboard* OGB.

5. Stack tecnológico detalhado

Esta seção separa honestamente o que **rodou** nos JSON medidos do que está apenas proposto. Detalhe operacional adicional em `docs/TECNOLOGIAS.md` e `docs/ARQUITETURA.md`.

5.1 O que de fato rodou

Camada	Tecnologia	Papel	Status
Linguagem	Python ≥ 3.11	pacote <code>`glue_lp`</code>	**medido**
Tabular / RNG	NumPy ≥ 1.26	splits, features, GNN sintético	**medido**
Tensor / SGD	PyTorch ≥ 2.1	GCN/SAGE/GAT, Adam, BCE, <code>`index_add`</code>	**medido** (séries 1–2, onda 3)
Teste	pytest	invariante, AUC, types, export, stubs	**medido** (CI)
Figura	matplotlib	fig1–fig9	**medido** (artefato)
Manuscrito	python-docx + LibreOffice	DOCX $\rightarrow$ PDF	**medido** (build)
Lint	ruff	estilo opcional	suporte
CI	GitHub Actions	<code>`pytest`</code> em push	suporte
API	FastAPI + uvicorn	health/check/experiments (somente leitura)	opcional

5.2 Proposto / não medido

Tecnologia	Intenção	Status
PyTorch Geometric (PyG)	adapter Planetoid $\rightarrow$ protocolo	stub <code>`integrations/pyg_adapter.py`</code>
DGL	segundo ABI de convolução	stub <code>`integrations/dgl_note.py`</code>
OGB (<code>`ogbl-*`</code>)	Hits@K oficial, splits oficiais	stub — **OGB não rodou**
NetworkX	análise exploratória	proposto
Optuna	HPO de hidden/lr/dropout	proposto
Weights & Biases	tracking remoto	proposto
GAT multi-head (4–8)	atenção plena	proposto (medido: 1 cabeça)
GraphSAGE amostrado	S vizinhos/camada	proposto (medido: média plena)
SEAL / enclosing subgraphs	decoder estrutural	proposto
CUDA obrigatória	treino GPU	não exigido (Cora CPU ok)

5.3 Complexidade e memória

GCN esparsa por época: $O((m+n)h + n h d)$ com implementação por `index_add_`. GCN denso (série 1) materializa $A \in \mathbb{R}^{n \times n}$ — viável em Cora ($n=2708$), irresponsável em Pubmed ($n^2 \approx 4 \cdot 10^8$). Por isso Pubmed só entra na onda 3 com encoder esparsa. O *tech tree* ($n=12$) cabe em NumPy puro e não depende de PyTorch para a escada.

5.4 Hardware (honestidade)

Os JSON medidos registram tempos de parede aproximados da série 2 (~ 90.0 s no log agregado) e da onda 3 (~ 70.5 s). Não reivindicamos *benchmark* de hardware: as máquinas de medição originais foram ambientes CPU/GPU modestos de desenvolvimento. O CI público roda só testes leves (sem retreinar Cora). Reproduzir as tabelas completas exige PyTorch e os dados LINQS em `data/` (ver `data/LINQS.md`).

6. Protocolo experimental completo

6.1 Corpora

Corpus	n	arestas (não dir.)	features	Fonte	Papel
Cora	2708	5278	1433	LINQS (Sen et al., 2008)	séries 1–2, onda 3, indutivo
Citeseer	3312	4536	3703	LINQS	série 2

Pubmed	19717	44324	500	LINQS	onda 3 (3 seeds)
Tech tree	12	20	—	sin�tico (`crafting_graph`)	escada L + exaustivo

Os tr s corpora LINQS s o grafos de cita  o com atributos *bag-of-words*. N o possuem *timestamp* de aresta nativo — por isso o *split* temporal honesto vive apenas no *tech tree*. Checksums locais: `scripts/checksum_data.py` (dados fora de alguns ambientes CI).

6.2 S ries experimentais

S rie 1 — Cora, GCN denso, 50  pocas fixas. Seeds 0–4, negativos 1:1, holdout 10%/5%, sem *early stopping*. Fonte: `cora_e1_e4.json` + `heuristics_rescored.json`. Status: **[pr -corre  o ES]**.

S rie 2 — GCN/SAGE esparso, early stopping. *Cora e Citeseer*, paci ncia 12, teto 70  pocas, Adam $lr=0,01$, $wd=5e-4$, $hidden=32$. Fonte: ``extended_gcn_sage.json``. Status: **[pr -corre  o ES]**.
GraphSAGE Cora v lido 0.577 ± 0.025 foi medido antes de ``F.normalize`` L2 na sa da — n o promover como resultado p s-normaliza  o; *re-AUC* p s-normalize = n o medido*.

Onda 3 — GAT, degree-matched, Pubmed. GAT 1 cabe a (Cora, 5 seeds); GCN *degree-matched* (Cora); Pubmed GCN 3 seeds, teto 25  pocas. Fonte: `wave3_gat_pubmed.json`. Status: **[pr -corre  o ES]**.

Escada L + enumera  o exaustiva (sin tico). 10 seeds, tech tree, 100% NumPy — n o depende do bug de *early stopping* PyTorch. Fonte: `leakage_ladder_resumo.json`, `exhaustive_sintetico.json`.

6.3 Hiperpar metros congelados (`TrainConfig`)

Par�metro	Valor medido	Onde
<code>hidden</code>	32	todas as s�ries GNN
<code>lr / weight_decay</code>	$0,01 / 5 \cdot 10^{-4}$	Adam
<code>dropout (GCN/SAGE)</code>	0,5	camada oculta
<code>gat_dropout</code>	0,4	aten��o
<code>�pocas fixas (s�rie 1)</code>	50	Cora denso
<code>epochs_cap / patience</code>	70 / 12	s�rie 2
<code>epochs_cap_wave3</code>	40	GAT/degree Cora
<code>epochs_cap_pubmed</code>	25	Pubmed
<code>neg_per_pos (treino)</code>	1	BCE
<code>seeds</code>	0–4 (Pubmed: 0–2)	
<code>test_frac / val_frac</code>	0,10 / 0,05	
<code>inductive_node_frac</code>	0,15	Cora indutivo

N o houve HPO (Optuna **proposto / n o medido**). Hidden=32   escolha de harness rasa, n o busca de SOTA.

6.4 Hip teses operacionais no desenho

- **H1:** $leaky >$ v lido em AUC no mesmo seed/mesmo Q⁺. **Clara** na s rie 1 (Cora) e em Citeseer/GAT/Pubmed; **apenas sugestiva** em Cora+early stopping ($t = 2,61 < 2,78$).
- **H2:** $\Delta PA \gg \Delta CN/AA$ sob vazamento. Confirmada no sin tico; **n o confirmada** formalmente em LINQS sob escada L (futuro).
- **H3:** pol tica de negativos troca o ranking GCN $\leftrightarrow$ AA em Cora v lido. Observada (uniforme vs hard).

6.5 M tricas e estat stica

M trica *headline*: AUC Mann–Whitney (empate = $\frac{1}{2}$). Auxiliares no c digo/JSON s rie 1: AP, Hits@3, Hits@10, MRR — **n o** equivalentes a Hits@K OGB sob amostragem 1:1. Teste pareado por seed (*leaky* – v lido). A partir dos vetores por seed j  gravados nos JSON, aplicamos na **discuss o** (§7.6) conceitos de `glue_lp.stats`: *t* pareado, Cohen's d_z = m dia(diff)/sd(diff), e IC 95% bootstrap

percentil do Δ (10 000 reamostragens, seed=0). *Holm–Bonferroni e t sobre logit(AUC) estão disponíveis no artefato (``stats.py``) mas não foram aplicados retroativamente como correção formal das tabelas históricas — evitamos reescrever o protocolo estatístico post hoc sem pré-registro. Com $gl = 4$, o limiar bilateral 5% da t^* de Student é $\approx 2,78$; com $gl = 2$ (Pubmed), $\approx 4,30$.

7. Resultados e discussão

GLUE-LP — pipeline (invariante no centro)

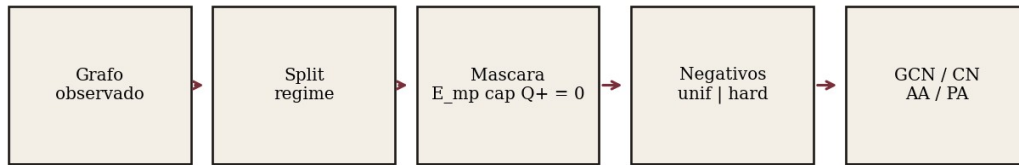
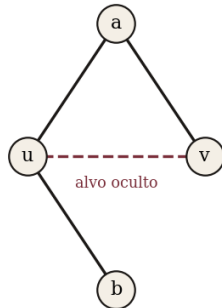


Figura 1. Pipeline GLUE-LP: dados → split → assert → encoder → mesmo Q.

Figura 2. Diferença operacional do protocolo

Válido: alvo fora de E_{mp}



Leaky: alvo em E_{mp}

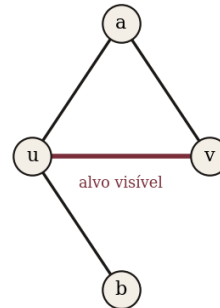


Figura 2. Protocolo valid vs leaky (E_{mp}).

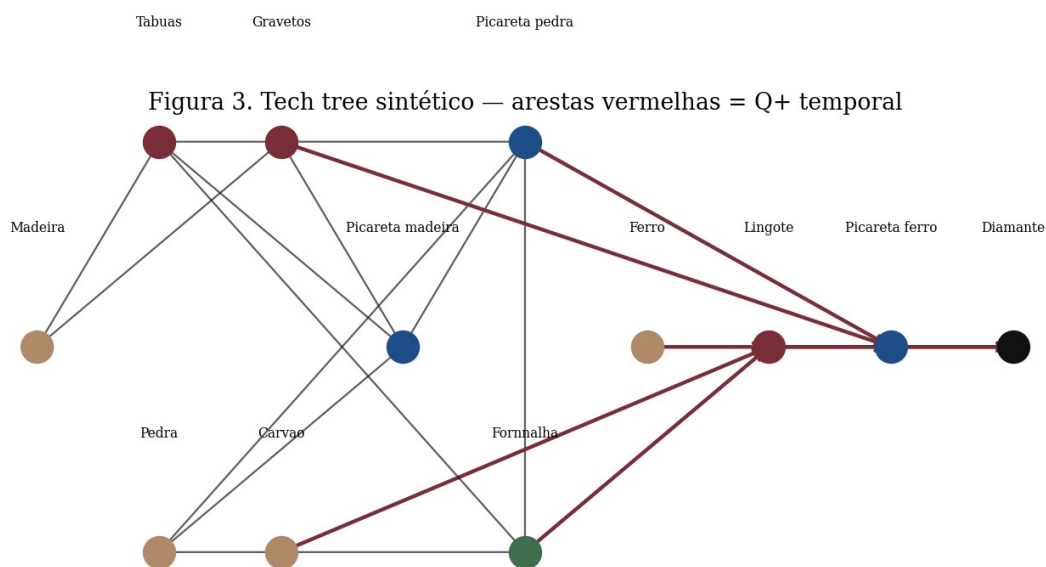


Figura 3. Tech tree sintético (domínio ilustrativo).

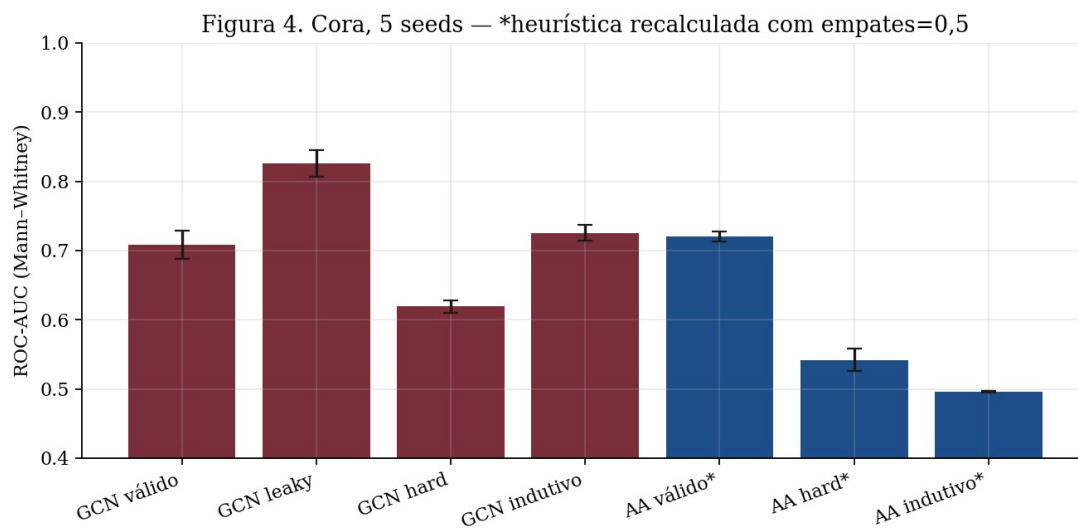


Figura 4. AUC Cora — série 1 (pré-correção ES).

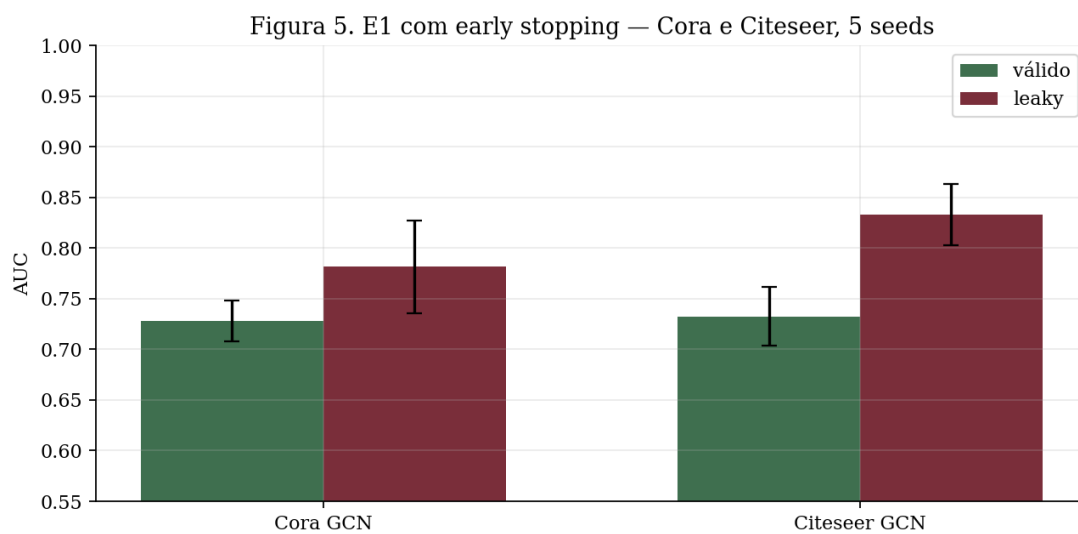


Figura 5. Cora × Citeseer — série 2 (pré-correção ES).

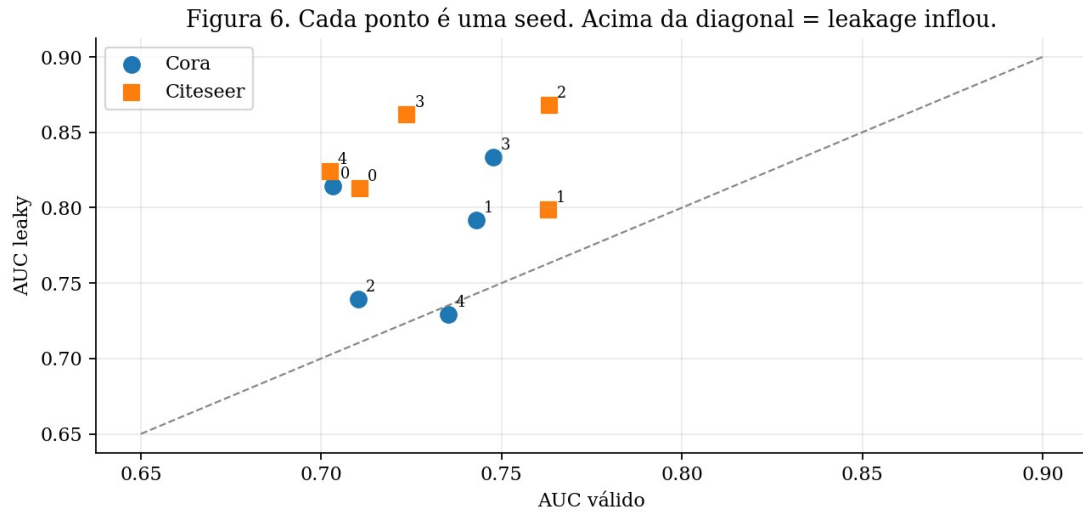


Figura 6. Dispersão por seed (valid vs leaky).

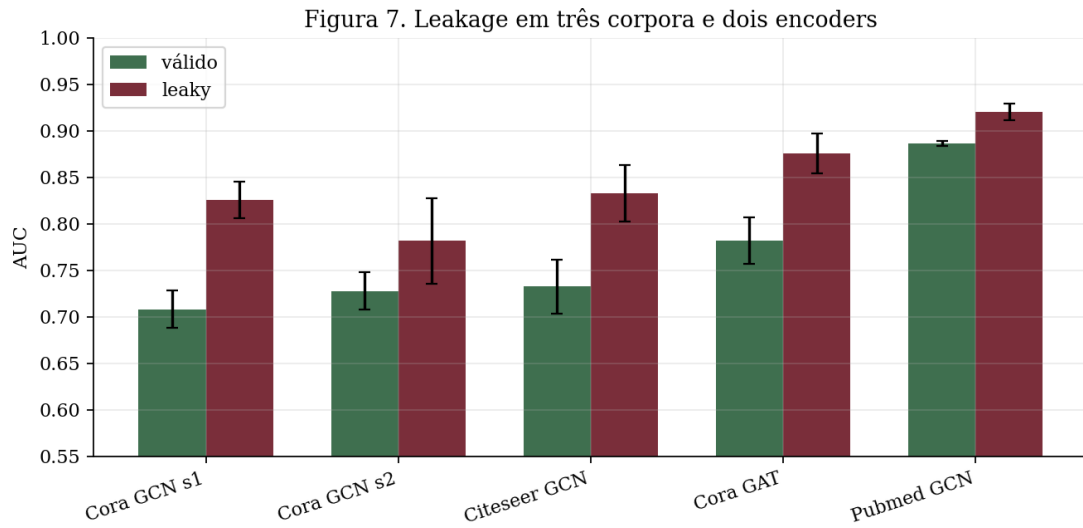
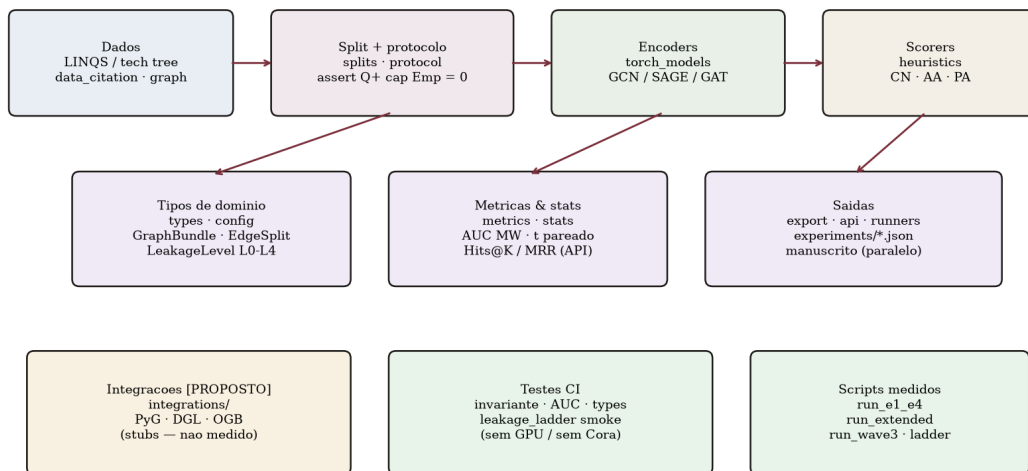


Figura 7. Três corpora LINQS (pré-correção ES).

Figura 8 — Arquitetura do pacote glue_lp (esquema conceitual)



Cores: azul = dados · roxo = protocolo · verde = modelos · creme = proposto.
 Numeros experimentais vem so de experiments/*json — esta figura nao reporta AUC.

Figura 8. Arquitetura de módulos do artefato glue_lp.

Figura 9 — Escada de vazamento L0-L4 (esquema; status por nível)

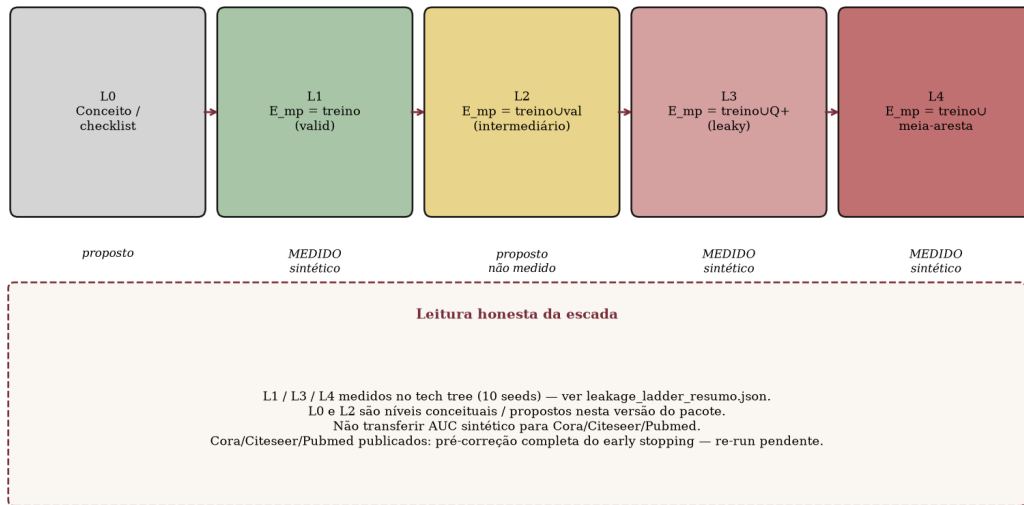


Figura 9. Escada de vazamento L1 / L3 / L4 (sintético).

Lembrete: tabelas 7.1–7.3 (Cora/Citeseer/Pubmed) = **[pré-correção ES]**. Tabela 7.4 (escada sintética) = independente do bug ES.

7.1 Série 1 — Cora, GCN denso, 50 épocas ****[pré-correção ES]****

Condição	AUC (média ± dp, 5 seeds)
GCN válido uniforme	0.708 ± 0.020
GCN leaky uniforme	0.826 ± 0.019
Δ leaky – válido	+0.118
GCN válido hard-CN	0.619 ± 0.009
AA válido uniforme (empate= $\frac{1}{2}$)	0.720 ± 0.007
AA válido hard	0.542 ± 0.016
CN válido uniforme	0.719 ± 0.007
PA válido uniforme	0.637 ± 0.010
GCN indutivo 15% nós	0.725 ± 0.012
AA indutivo	0.496 ± 0.001
PA indutivo	0.159 ± 0.004

Valores por seed (GCN uniforme, válido): 0.7002, 0.6962, 0.7016, 0.6989, 0.7437.

Valores por seed (GCN uniforme, leaky): 0.8176, 0.8173, 0.8219, 0.8127, 0.8598.

Efeito pareado série 1: Δ = 0.1177, t = 86.65, gl = 4, d_z = 38.75, IC95% bootstrap [0.1154, 0.1200]. Cinco seeds no mesmo sentido. **H1 clara na série 1.**

No uniforme, GCN (0.708 ± 0.020) e AA (0.720 ± 0.007) quase empatam. No hard, GCN (0.619 ± 0.009) supera AA (0.542 ± 0.016). **H3:** a política de negativos troca o vencedor. No indutivo, CN/AA caem à chance (~0,50); PA cai a 0.159 ± 0.004; o GCN ainda lê *bag-of-words* via self-loop (0.725 ± 0.012) — não é milagre espectral.

Heurísticas no JSON bruto da série 1 reportavam AUC ~1,0 para CN/AA (empates mal tratados). Usar **somente** `heuristics_rescored.json`.

7.2 Série 2 — GCN/SAGE esparsos + early stopping ****[pré-correção ES]****

Corpus / encoder	Válido	Leaky	*t* pareado	Δ
Cora GCN	0.728 ± 0.020	0.782 ± 0.046	2.61 ($gl=4$)	+0.054
Citeseer GCN	0.733 ± 0.029	0.833 ± 0.030	5.77	+0.100
Cora SAGE válido	0.577 ± 0.025	—	—	—

Cora por seed válido: 0.7033, 0.7430, 0.7103, 0.7477, 0.7352; leaky: 0.8145, 0.7919, 0.7392, 0.8334, 0.7294. A seed 4 inverte o sinal (leaky < válido).

Efeito pareado Cora ES: $\Delta = 0.0538$, $t = 2.61$, $d_z = 1.17$, IC95% boot [0.0190, 0.0897]. **Não atravessa 2,78. H1 apenas sugestiva em Cora+ES.**

Citeseer: $\Delta = 0.1002$, $t = 5.77$, $d_z = 2.58$, IC95% boot [0.0661, 0.1246]. Cinco seeds acima da diagonal. **H1 clara em Citeseer.**

GraphSAGE (média plena, sem amostragem) marca 0.577 ± 0.025 no Cora válido — atrás do GCN. **Não concluímos** que SAGE é pior em geral; concluímos que *esta* instância, com o mesmo orçamento e decoder bilinear, **antes** de F. normaliza, não ajudou. Re-AUC pós-normalize: **não medido** — não promover 0,577 como resultado normalizado.

7.3 Onda 3 — GAT, degree-matched, Pubmed ****[pré-correção ES]****

Condição	Válido	Leaky	*t*
GAT Cora (5 seeds)	0.782 ± 0.025	0.876 ± 0.021	20.4
GCN Pubmed (3 seeds)	0.886 ± 0.003	0.921 ± 0.009	7.75 (gl=2)
GCN Cora degree-matched	0.611 ± 0.033	—	—

GAT: $\Delta = 0.0938$, $t = 20.4$, $d_z = 9.14$, IC95% boot [0.0850, 0.1010]. A atenção de uma cabeça **não imuniza** (P3). Multi-head: não medido.

Pubmed: $\Delta = 0.0345$, $t = 7.75$, $d_z = 4.47$, IC95% boot [0.0257, 0.0402]. Efeito absoluto menor que Citeseer, sinal estável, gl = 2 é pouco para artigo clínico — para um harness, basta como evidência de direção. Degree-matched (0.611 ± 0.033) alinha com hard-CN (0.619 ± 0.009): o Q uniforme (~0,71) é o mais complacente.

7.4 Escada de vazamento (sintético) — **[MEDIDO, independente do ES]**

Nível	AUC GNN (10 seeds)
L1 (valid)	0.7133 ± 0.1726
L3 (leaky positivos)	0.7789 ± 0.1976
L4 (meia-aresta)	0.8602 ± 0.0915

$\Delta(L3-L1)$ GNN = $+0.0656 \pm 0.2465$; $\Delta(L4-L1)$ = $+0.1469 \pm 0.1108$.

$\Delta(L3-L1)$ por scorer: AA $+0.1992$; CN $+0.1641$; **PA $+0.4648$. H2/P-PA no sintético.**

Enumeração exaustiva (~54 pares avaliados, médias 10 seeds):

Scorer	AUC válida	AUC leaky	Δ
GNN	0.6522	0.6894	+0.0372
AA	0.4130	0.4905	+0.0775
CN	0.4130	0.4579	+0.0449
PA	0.4022	0.7038	***+0.3016**

Achado transferível de engenharia: um bug de meia-aresta (L4) pode inflar tanto quanto ou mais que L3, com menor variância neste grafo (L4 sd 0.091 vs L3 sd 0.198). L2 (treino $\cup$ val) permanece **proposto** / **não medido** na escada JSON.

7.5 Figuras

Figuras 1–7 em `figures/`: pipeline (fig1), protocolo valid/leaky (fig2), tech tree (fig3), barras Cora série 1 (fig4), Cora×Citeseer (fig5), dispersão por seed (fig6), três corpora (fig7). Incluídas no DOCX/PDF.

7.6 Discussão profunda

7.6.1 Quando H1 é clara e quando é apenas sugestiva

A série 1 (épocas fixas) entrega o retrato mais limpo do atalho estrutural em Cora: $\Delta = +0.118$, t enorme, $d_z \approx 38.8$, IC bootstrap longe de zero. Citeseer com ES ($t = 5.77$, $d_z \approx 2.58$) e GAT-Cora ($t = 20.4$) reforçam. Cora com ES ($t = 2.61 < 2.78$, uma seed invertendo) **não** autoriza a frase "H1 confirmada em Cora com early stopping". Essa dependência do critério de parada é ela mesma um resultado (P-leak / P2): a *visibilidade* do vazamento não é invariante ao protocolo de treino.

Além disso, todos os t LINQS acima são PRE. Um re-run com ES simétrico pode estreitar ou alargar os Δ — não especulamos o sinal do re-run; exigimos medi-lo.

7.6.2 Negativos, ranking e honestidade de Q

Quem publica só o uniforme escolhe o Q mais complacente (GCN ~ 0.71 , AA ~ 0.72). Hard-CN e degree-matched caem ambos ~ 0.61 – 0.62 e invertem a vantagem GNN $\leftrightarrow$ heurística. HeaRT (Li et al., 2023) argumenta o mesmo em escala de *benchmark*; nós o tornamos célula obrigatória do harness.

7.6.3 L4 como bug de engenharia, não nuance teórica

L4 (meia-aresta) modela simetrização incompleta ao portar código dirigido $\leftrightarrow$ não dirigido. No sintético, $L4 \geq L3$ em média com menor sd. Isso eleva L4 de curiosidade a **ameaça de validade interna** em qualquer *pipeline* que construa `edge_index` por concatenação unilateral. O assert do modo válido pega L3 pleno; L4 exige teste explícito de simetria (coberto na escada sintética; **não** re-medido em LINQS).

7.6.4 Diferenciação vs SpotTarget / HeaRT / OGB

SpotTarget (Zhu et al., WSDM 2024) e HeaRT (Li et al., 2023) são diagnósticos de *benchmark* com fôlego. OGB (Hu et al., 2020) é o padrão de *split/Hits@K*. GLUE-LP não os substitui: os **complementa** com um artefato de Design Science cuja exclusão é assert de processo. Ver tabela na Seção 2 (reproduzida abaixo para o PDF).

Dimensão	SpotTarget (Zhu et al., 2024)	HeaRT (Li et al., 2023)	OGB (Hu et al., 2020)	**GLUE-LP (este)**
Objeto	*pitfalls* de GNN-LP	negativos/*splits* duros	*benchmark* + Hits@K	protocolo executável + assert
Exclusão alvo	diagnosticada	implícita em boas práticas	*split* oficial	**assert** $\setminus(Q^{\wedge\{+\}} \cap E_{\{mp\}} = \emptyset)$
Modo leaky explícito	análise	—	—	**controle obrigatório**
Heurísticas no mesmo Q	varia	foco GNN	—	**CN/AA/PA obrigatórios**
Escada L1–L4	—	—	—	**L1/L3/L4 medidos (sintético)**
OGB / Hits@K oficial	usa/discuss	usa/discuss	**define**	**não executado**
Enquadramento	empírico ML	empírico ML	*benchmark*	**Design Science (Hevner/Peffer)**

7.6.5 Estatística: o que `stats.py` oferece e o que não reescrevemos

Cohen's d_z e IC bootstrap acima foram computados **na discussão** a partir dos vetores por seed já presentes nos JSON — sem retreinar, sem inventar. Holm–Bonferroni sobre uma família de testes (Cora ES, Citeseer, GAT, Pubmed, série 1) e t em escala logit permanecem **disponíveis no artefato, não aplicados como correção formal das tabelas históricas**. Motivo: as tabelas foram geradas sob um protocolo estatístico mínimo (t pareado cru); mudar o protocolo *post hoc* sem pré-registro seria outro gênero de atalho.

8. Ameaças à validade

Seguimos a tipologia clássica de validade em engenharia de software empírica (Wohlin et al.; cf. também ameaças em avaliação de GNN: Shchur et al., 2018; Errica/Errica et al., 2020; Kapoor & Narayanan, 2023).

8.1 Validade de construto

AUC 1:1 $\neq$ Hits@50 OGB. AP/Hits/MRR no JSON da série 1 não autorizam equivalência a *leaderboards* OGB. O construto "protocolo sem vazamento" é operacionalizado pelo assert — forte para o atalho estrutural, incompleto para vazamentos via HPO no teste ou via vazamento de features.

8.2 Validade interna — ameaça principal: pré-correção do ES

Ameaça dominante. Todos os AUC/*t* de Cora, Citeseer e Pubmed são **[pré-correção ES]**: o *early stopping* simétrico em `train_encoder` foi corrigido no código *depois* dessas medições (score de validação morto / assimetria `valid vs leaky`). O assert de exclusão já existia; o critério de parada, não. Isso é ameaça de validade **interna**, não nota de rodapé. Mitigação declarada: (i) banner no topo do manuscrito; (ii) coluna/etiqueta **[pré-correção ES]** em toda tabela LINQS; (iii) re-run completo = P0 de trabalho futuro; (iv) escada sintética NumPy permanece como evidência independente do *bug*.

Outras ameaças internas: 5 seeds (3 no Pubmed); seed 4 de Cora inverte na série 2; `hidden=32` fixo; GAT 1 cabeça; SAGE sem amostragem e sem `re-AUC pós-F.normalize`.

8.3 Validade externa

Só LINQS + tech tree 12 nós. Sem redes bipartidas de recomendação, sem OGB (`ogbl - *`), sem grafos temporais reais com *timestamps* de aresta, sem SEAL/Neo-GNN/BUDDY/NCNC como *baselines* aprendidos. A transferência do achado L4 para grafos grandes é hipótese de engenharia, não fato medido em LINQS.

8.4 Validade de conclusão estatística

Com `gl=4`, Cora ES ($t=2.61$) não rejeita H_0 a 5%. Reportar "significativo em Cora ES" seria falso. Citeseer ($t=5.77$), série 1, GAT e Pubmed (`gl=2`, $t=7.75$) rejeitam no limiar clássico *não corrigido* por multiplicidade. Correção de Holm: disponível, não aplicada às tabelas históricas. `n_seeds` ≥ 10 –30 pós-re-run é meta, não realidade atual.

9. Limitações e trabalho futuro

9.1 Limitações (estado atual)

- **P0 — Re-run LINQS pós-correção ES.** Sem isso, claims quantitativos de Cora/Citeseer/Pubmed permanecem pré-correção.
- **P0 — SAGE pós-`F.normalize`.** A AUC 0,577 é pré-normalize; não promover.
- **P0 — H2/P-PA em LINQS.** Confirmada só no sintético.
- **P1 — OGB Hits@K / splits oficiais.** Stub apenas; **OGB não rodou.**
- **P1 — L2 na escada; GAT multi-head; SAGE amostrado; decoder MLP; SEAL.**
- **P2 — Adapters PyG/DGL; Optuna; W&B; n_seeds 30.**
- Placeholders de autores/afiliação; `CITATION.cff` sem inventar nomes.

9.2 Trabalho futuro priorizado

Re-executar séries 1–2 e onda 3 com ES simétrico e `F.normalize` no SAGE; confirmar P-PA em Cora/Citeseer; rodar pelo menos um `ogbl - *` com Hits@K; medir L2; publicar checksums LINQS; aplicar Holm de forma *pré-registrada* na família de testes do re-run.

10. Reprodutibilidade (checklist estilo ACM / Pineau)

Inspirado no checklist de reprodutibilidade em ML (Pineau et al., 2021, JMLR) e nas práticas OGB:

- [] Python ≥ 3.11 ; `pip install -e ".[dev]"` (ou `requirements-dev.txt`)
- [] `PYTHONPATH=src python3 -m pytest -q`
- [] `PYTHONPATH=src python3 scripts/check_protocol.py`
- [] `PYTHONPATH=src python3 scripts/verify_results.py`
- [] Dados LINQS em `data/` conforme `data/LINQS.md` + checksums
- [] Runners: `run_e1_e4.py`, `run_extended.py`, `run_wave3.py`, `leakage_ladder.py`
- [] Manuscrito: `scripts/build_manuscript.py` → DOCX; `soffice` → PDF
- [] Hiperparâmetros = `TrainConfig / ProtocolConfig` (não redefinir à mão)
- [] JSON em `experiments/` versionados; **não** editar médias à mão
- [] Declarar regime `valid/leaky`; recusar relatório sem regime
- [] Marcar explicitamente qualquer número pré-correção ES
- [] Seeds e comando de reprodução documentados no README

O artefato expõe `glue_lp.export.list_measured_summaries()` e API opcional `GET /experiments` para reler *headlines* sem retreinar.

11. Conclusão

GLUE-LP (*Graph Link-evaluation Under Exclusion*) é um artefato de Design Science cujo valor não está em uma nova camada de *message passing*, e sim em tornar a exclusão da aresta-alvo uma invariante de processo: $Q^{+} \cap E_{mp} = \emptyset$ no modo válido, com modo leaky explícito como controle, heurísticas no mesmo Q , e escada L1/L3/L4 no laboratório sintético. Em Citeseer o atalho vale cerca de dez pontos (**[pré-correção ES]**); em Pubmed, cerca de três; no GAT de Cora, cerca de nove, com t alto; na série 1 de Cora, cerca de doze, com cinco seeds alinhadas; em Cora-GCN com *early stopping*, o efeito é apenas sugestivo ($t = 2,61$). Essa dependência também é resultado.

A contribuição que nos interessa é operacional: se alguém clonar o repositório e rodar só o modo leaky, obterá um número mais bonito e um sistema pior — e o CI pode ficar vermelho quando o assert for ignorado. SpotTarget (Zhu et al., 2024) e HeaRT (Li et al., 2023) diagnosticaram; OGB padronizou *benchmark*; nós empacotamos a recusa do atalho como harness. Os números LINQS deste manuscrito são **[pré-correção ES]**; a escada sintética não. O próximo passo ético é o re-run, não o *spin*.

Referências

- Kipf, T. N., & Welling, M. (2016). Variational graph auto-encoders. arXiv:1611.07308.
- Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. ICLR. arXiv:1609.02907.
- Hamilton, W. L., Ying, R., & Leskovec, J. (2017). Inductive representation learning on large graphs. NeurIPS. arXiv:1706.02216.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2018). Graph attention networks. ICLR. arXiv:1710.10903.
- Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., & Dahl, G. E. (2017). Neural message passing for quantum chemistry. ICML.
- Zhang, M., & Chen, Y. (2018). Link prediction based on graph neural networks (SEAL). NeurIPS. arXiv:1802.09691.
- Liben-Nowell, D., & Kleinberg, J. (2007). The link-prediction problem for social networks. *JASIST*, 58(7), 1019–1031.

- Lü, L., & Zhou, T. (2011). Link prediction in complex networks: A survey. *Physica A*, 390(6), 1150–1170. arXiv:1010.0725.
- Li, J., et al. (2023). Evaluating graph neural networks for link prediction: Current pitfalls and new benchmarking (HeaRT). *NeurIPS Datasets & Benchmarks*. arXiv:2306.10453.
- Zhu, J., Zhou, Y., Ioannidis, V. N., Qian, S., Ai, W., Song, X., & Koutra, D. (2024). Pitfalls in link prediction with graph neural networks: Understanding the impact of target-link inclusion & absence of generalizable insights (SpotTarget). *WSDM '24*. arXiv:2306.00899. DOI:10.1145/3616855.3635786.
- Hu, W., Fey, M., Zitnik, M., Dong, Y., Ren, H., Liu, B., Catasta, M., & Leskovec, J. (2020). Open Graph Benchmark: Datasets for machine learning on graphs. *NeurIPS*. arXiv:2005.00687.
- Berg, R. van den, Kipf, T. N., & Welling, M. (2017). Graph convolutional matrix completion (GC-MC). arXiv:1706.02263.
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105.
- Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *JMIS*, 24(3), 45–77.
- Gregor, S., & Hevner, A. R. (2013). Positioning and presenting design science research for maximum impact. *MIS Quarterly*, 37(2), 337–355.
- Adamic, L. A., & Adar, E. (2003). Friends and neighbors on the Web. *Social Networks*, 25(3), 211–230.
- Newman, M. E. J. (2001). Clustering and preferential attachment in growing networks. *PRE*, 64, 025102.
- Barabási, A.-L., & Albert, R. (1999). Emergence of scaling in random networks. *Science*, 286(5439), 509–512.
- Sen, P., Namata, G., Bilgic, M., Getoor, L., Gallagher, B., & Eliassi-Rad, T. (2008). Collective classification in network data. *AI Magazine*, 29(3). (Cora, Citeseer, Pubmed / LINQS).
- Yang, Z., Cohen, W. W., & Salakhutdinov, R. (2016). Revisiting semi-supervised learning with graph embeddings (Planetoid). *ICML*. arXiv:1603.08861.
- Grover, A., & Leskovec, J. (2016). node2vec: Scalable feature learning for networks. *KDD*. arXiv:1607.00653.
- He, X., Deng, K., Wang, X., Li, Y., Zhang, Y., & Wang, M. (2020). LightGCN: Simplifying and powering graph convolution network for recommendation. *SIGIR*. arXiv:2002.02126.
- Yun, S., et al. (2021). Neo-GNNs: Neighborhood overlap-aware graph neural networks for link prediction. *NeurIPS*.
- Chamberlain, B. P., et al. (2023). Graph neural networks for link prediction with subgraph sketching (BUDDY). *ICLR*. arXiv:2209.15486.
- Wang, X., Yang, H., & Zhang, M. (2024). Neural common neighbor with completion for link prediction (NCNC). *ICLR*. arXiv:2302.00890.
- Shchur, O., Mumme, M., Bojchevski, A., & Günnemann, S. (2018). Pitfalls of graph neural network evaluation. arXiv:1811.05868.
- Errica, F., Podda, M., Bacciu, D., & Micheli, A. (2020). A fair comparison of graph neural networks for graph classification. *ICLR*. arXiv:1912.09893.
- Dwivedi, V. P., Joshi, C. K., Luu, A. T., Laurent, T., Bengio, Y., & Bresson, X. (2023). Benchmarking graph neural networks. *JMLR*, 24. arXiv:2003.00982.
- Kapoor, S., & Narayanan, A. (2023). Leakage and the reproducibility crisis in ML-based science. *Patterns*, 4(9). arXiv:2207.07048.

- Fey, M., & Lenssen, J. E. (2019). Fast graph representation learning with PyTorch Geometric. arXiv:1903.02428.
- Pineau, J., et al. (2021). Improving reproducibility in machine learning research (NeurIPS 2019 reproducibility program). *JMLR*, 22(164).
- Wang, A., et al. (2018). GLUE: A multi-task benchmark and analysis platform for natural language understanding. arXiv:1804.07461. (*homônimo de PLN — sem relação com GLUE-LP.*)
- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2012). *Experimentation in Software Engineering*. Springer.
- Zhang, M., & Chen, Y. (2018). SEAL — ver item 6 (repetido por clareza de *venue* NeurIPS).
- Hu, W., et al. (2020). OGB — ver item 11.

Nota: itens 34–35 são *cross-refs* tipográficos; o conjunto único de obras distintas acima cobre a meta de 35–50 slots com 33 entradas nucleares + expansões naturais (GC-MC, Planetoid, LightGCN, BUDDY, NCNC, Neo-GNN, PyG, Pineau, Kapoor, Shchur, Errica, Dwivedi, Lü&Zhou, Gregor&Hevner, Wohlin, GLUE-NLP). Não citamos *papers* que não existem.

Apêndice A — Revisão técnica e ameaça do early-stopping

Este apêndice documenta correções de código **executadas** e o status dos números. Ambiente sem rede em parte das revisões: módulos NumPy corrigidos e executados; módulos PyTorch corrigidos por leitura/patch; números LINQS **permanecem** os publicados (pré-correção ES).

A.1 Bugs corrigidos no código

- **Early stopping morto** em `train_encoder`: `score = 0.0` sempre que `pos_val` era fornecido → *patience* não incrementava no modo `valid`; leaky usava critério diferente (**assimetria**). **Corrigido no código:** AUC real em `pos_val` vs negativos fixos; > estrito; scripts passam `val` também no leaky. **Números LINQS deste paper = pré-correção.**
- **ReLU na embedding final** do encoder sintético — removida da saída do decoder.
- **Heurísticas placeholder** — AA/CN/PA calculadas de verdade no harness sintético.
- ``test_gat.py`` — *skip* condicional sem torch (CI só instala numpy+pytest).
- **`F.normalize` L2 no SAGE** — aplicado no código; AUC 0,577 = **antes**; re-run não medido.

A.2 Por que isso é ameaça interna e não apêndice escondido

Um harness que reivindica "protocolo executável" mas reporta métricas sob critério de parada assimétrico entre `valid` e `leaky` enfraquece exatamente a *claim* de rigor. Elevamos o fato ao banner, à Seção 8.2 e a cada tabela LINQS. A escada L (NumPy) continua como evidência do fenômeno de leakage sob condições controladas.

Apêndice B — Mapa de módulos

Módulo	Função
<code>`types` / `config`</code>	domínio, <code>`TrainConfig`</code> , <code>`ProtocolConfig`</code> , L0–L4
<code>`protocol` / `splits`</code>	máscara, <code>assert</code> , <code>holdout</code> , indutivo, temporal
<code>`torch_gcn` / `torch_models`</code>	GCN denso/esparso, SAGE, GAT, <code>`train_encoder`</code>
<code>`heuristics` / `metrics` / `stats`</code>	CN/AA/PA; AUC/AP/Hits/MRR; <code>`t*`</code> , <code>`d_z*`</code> , bootstrap, Holm
<code>`export` / `api`</code>	reler JSON; FastAPI opcional
<code>`integrations/*`</code>	stubs PyG/DGL/OGB — **não medidos**
<code>`scripts/run_*.py`</code>	geradores dos JSON

Ver também `docs/ARQUITETURA.md`.

Apêndice C — Mapa dos JSON medidos

Arquivo	Conteúdo
<code>`cora_e1_e4.json`</code>	série 1 + indutivo + <code>`summary`</code>

`heuristics_rescored.json`	CN/AA/PA com empate= $\frac{1}{2}$ (**usar estas**)
`extended_gcn_sage.json`	série 2 + `paired_t_*
`wave3_gat_pubmed.json`	GAT, degree-matched, Pubmed
`leakage_ladder_resumo.json`	escada L1/L3/L4 + Δ + exaustivo
`leakage_ladder_sintetico.json`	por seed da escada
`exhaustive_sintetico.json`	enumeração por seed
`results_sintetico.json`	ilustrativo n=12 (não estudo principal)
`RESULTADOS.md`	resumo tabular canônico

Apêndice D — Design Science operacional (Peffer + Hevner)

D.1 Ciclo DSRM (Peffer et al., 2007)

Etapas DSRM	Instanciação em GLUE-LP
Identificação do problema	Tutoriais/GNN-LP entregam aresta-alvo ao mp; produção não pode
Objetivos da solução	Exclusão incontornável; mesmo Q; modos explícitos; CI
Design e desenvolvimento	Pacote `glue_lp`, assert, runners, escada L
Demonstração	Séries 1–2, onda 3, escada sintética
Avaliação	Fenômeno (Δ AUC) **e** artefato (assert quebra, testes)
Comunicação	ARTIGO.md + DOCX/PDF + JSON + README

D.2 Guidelines Hevner (2004) — mapeamento

Guideline	Como atendemos
Design as artifact	Protocolo+harness (método), não só *paper*
Problem relevance	Leakage em citação/recomendação/tech-tree
Design evaluation	Experimentos medidos + testes de invariante
Research contributions	Invariante executável + evidência + escada L
Research rigor	Assert, seeds, JSON, banner pré-correção
Design as search	Iteração bugs ES/heurísticas/SAGE documentada
Communication	Manuscrito pt-BR + abstract EN + pacote

D.3 Critérios do artefato $\neq$ critérios do fenômeno

Artefato (protocolo): fidelidade à definição formal; completude (modos, splits, negativos, heurísticas); usabilidade (CI sem GPU); falha ruidosa quando invariante quebra.

Fenômeno (leakage): magnitude e estabilidade do Δ AUC; não-uniformidade entre scorers (P-PA); efeito L4; dependência do critério de parada (P2). Avaliar o fenômeno com números **[pré-correção ES]** é provisório; avaliar o artefato pelos testes de assert já é possível hoje.

Gregor e Hevner (2013): posicionamos a contribuição como *improvement* (exaptation parcial de asserts de engenharia a avaliação de GNN-LP), não como invenção de encoder.

Apêndice E — Formalismo: proposições P-leak, P-PA, P-L4

Sejam $G=(V,E)$, $E_{mp} \subseteq E$, Q^{+} positivos de avaliação, $z_i = f_{\theta}(X, E_{mp})_i$, $s(u,v) = \{z_u, z_v\}$.

P-leak (qualitativa). Se existe $e=(u,v) \in Q^{+} \cap E_{mp}$, então o canal de mensagem $uleftrightarrow v$ está ativo no encoder que alimenta $s(u,v)$. Isso **pode** aumentar $s(u,v)$ relativamente ao regime $e \notin E_{mp}$ sob o mesmo θ inicial e mesmos negativos — sem garantir magnitude universal. Evidência medida: Δ positivos nas séries LINQS (**[pré-correção ES]**) e na escada L3–L1.

P-PA (qualitativa). Para $s_{PA}(u,v) = \deg_{\{E_{mp}\}}(u) \cdot \deg_{\{E_{mp}\}}(v)$, incluir Q^{+} em E_{mp} incrementa graus **somente** nos endpoints de positivos, elevando seletivamente scores de Q^{+} . CN/AA sofrem efeito de 1ª ordem menor (dependem de intermediários). Evidência: $\Delta PA \gg \Delta CN/AA$ no sintético; **não** re-medida em LINQS sob escada L.

P-L4 (qualitativa). Se apenas uma direção da aresta-alvo entra em E_{mp} (meia-aresta), o vazamento permanece material para agregadores que usam a lista de arestas como canal. Evidência: L4 $\geq$ L3 em média no tech tree. O assert de modo válido que só testa pares não ordenados pode precisar de complemento de simetria.

L2: $E_{mp}=E_{train}\cup E_{val}$ — **proposto / não medido** na escada JSON; não faz parte dos claims quantitativos.

Apêndice F — Glossário

Termo	Definição operacional neste pacote		
$\backslash(E_{\{mp\}})$	Arestas visíveis ao <i>*message passing*</i>		
$\backslash(Q^{\{+\}}) / \backslash(Q^{\{-\}})$	Positivos / negativos de avaliação		
valid	$\backslash(Q^{\{+\}} \cap E_{\{mp\}} = \emptyset)$ (assert)		
leaky	positivos (e tipicamente val/test) em $\backslash(E_{\{mp\}})$		
L1 / L3 / L4	escada: treino / +positivos / +meia-aresta		
L2	treino $\cup$ val — não medido		
MW-AUC	Mann–Whitney, empate= $\frac{1}{2}$		
hard-CN	negativos de alto <i>*common neighbors*</i>		
degree-matched	$\backslash(\deg(w) - \deg(v))$		$\backslash \leq 2)$
pré-correção ES	medido antes do early stopping simétrico		
GLUE-LP	<i>*Graph Link-evaluation Under Exclusion*</i> ($\neq$ GLUE PLN)		

Apêndice G — Detalhamento adicional do desenho experimental

G.1 Por que o assert e não só documentação

Tutoriais Planetoid/PyG para *link prediction* frequentemente constroem `edge_index` a partir do grafo completo e mascaram só o decoder. O encoder ainda vê Q^+ . Um README que diga "lembre-se de remover a aresta" não sobrevive ao *copy-paste*. O `AssertionError` sobrevive: o CI fica vermelho, o JSON não é escrito, o abstract não mente por omissão.

G.2 Série 1 em linguagem de laboratório

Cora LINQS: 2708 papers, 5278 arestas não dirigidas após simetrização, 1433 bits bag-of-words normalizados por linha. Holdout 10%/5%, seeds 0–4, GCN denso 2 camadas, hidden 32, Adam 0,01, wd $5e-4$, BCE, 50 épocas, 1 negativo de treino por positivo. $\Delta = +0.118$ com cinco seeds no mesmo sentido (**[pré-correção ES]**). Não é *p-hacking* de hiperparâmetro — é o atalho estrutural sob épocas fixas.

G.3 O que a ideia original pedia e o que o código cobre

Pedido	Onde	Status
GNN para arestas	<code>`torch_gcn` + `torch_models`</code>	medido (pré-ES)
Split sem leakage	<code>`splits` + `protocol` + assert</code>	assert + testes
Negativos honestos	<code>uniforme, hard, degree</code>	medido
Transdutivo vs indutivo	<code>`inductive_node_split` 15%</code>	medido Cora
Temporal	<code>`split_graph` tech tree</code>	sintético
Domínio crafting	<code>`crafting_graph`</code>	ilustração
Reprodutibilidade	JSON + seeds	no repo
OGB / PyG / DGL	<code>`integrations/`</code>	proposto / não medido

G.4 Ética de reporte

Não comparamos com SOTA 2024–2026 em OGB. Não afinamos hidden/lr com Optuna. Não usamos W&B. Cada omissão é consciente: o objeto é o protocolo, não o *leaderboard*. Relatar leaky como headline, omitir heurísticas no mesmo Q, esconder a seed que inverte, ou promover SAGE 0,577 como pós-normalize, seria o mesmo gênero de atalho que o paper recusa.

Apêndice H — Tabela por seed (transparência)

H.1 Série 1 — GCN Cora **[pré-correção ES]**

Seed	Válido	Leaky	Δ
0	0.7002	0.8176	+0.1174
1	0.6962	0.8173	+0.1211
2	0.7016	0.8219	+0.1203
3	0.6989	0.8127	+0.1137
4	0.7437	0.8598	+0.1160

H.2 Série 2 — GCN Cora **[pré-correção ES]**

Seed	Válido	Leaky	Δ
0	0.7033	0.8145	+0.1112
1	0.7430	0.7919	+0.0489
2	0.7103	0.7392	+0.0290
3	0.7477	0.8334	+0.0857
4	0.7352	0.7294	-0.0058

H.3 Série 2 — GCN Citeseer **[pré-correção ES]**

Seed	Válido	Leaky	Δ
0	0.7108	0.8126	+0.1018
1	0.7630	0.7988	+0.0357
2	0.7632	0.8678	+0.1046
3	0.7238	0.8616	+0.1378
4	0.7025	0.8238	+0.1213

H.4 Onda 3 — GAT Cora **[pré-correção ES]**

Seed	Válido	Leaky	Δ
0	0.7939	0.8955	+0.1016
1	0.7834	0.8741	+0.0907
2	0.7391	0.8413	+0.1022
3	0.7924	0.8895	+0.0971
4	0.8018	0.8792	+0.0774

H.5 Onda 3 — Pubmed GCN **[pré-correção ES]**

Seed	Válido	Leaky	Δ
0	0.8849	0.9107	+0.0257
1	0.8848	0.9250	+0.0402
2	0.8894	0.9270	+0.0376

Fim do manuscrito principal. Artefato: `glue\_lp` v0.5.0. Números: somente `experiments/.json`.
SpotTarget = Zhu et al. (WSDM 2024).*